

CSE4502 (Operating Systems Lab): 1A

Jibon Naher¹ and Samnun Azfar²

¹Assistant Professor, CSE

²Junior Lecturer, CSE

January 27, 2026

Part A: Process Scheduling Test Case and CPU Time Slice Analysis

Objective: Demonstrate how different CPU scheduling algorithms allocate CPU time slices to processes.

Setup

Clone the following git repository:

<https://github.com/azfarus/cpu-scheduling-simulation.git>

Git Link for Scheduling Simulation.

Instructions

- Navigate to the `src/` and `include/` directories.
- You must fill up the boilerplate codes in the **Scheduler.hpp** and **Scheduler.cpp** files to implement the scheduling logic for the required algorithms.
- Pay close attention to **process.hpp** and **process.cpp**. These files contain the `Process` class/struct definition, which includes vital attributes such as Arrival Time (AT), Burst Time (BT), program counter, total instructions and Priority. These values are essential for calculating the execution order and priorities in your implementation.

Test Case (5 Processes)

Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	4	1
P3	2	2	3
P4	3	6	2
P5	4	3	4

Note: Lower priority number means higher priority.

1. FCFS Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+-----+-----+-----+-----+-----+
|   P1   |   P2   |   P3   |   P4   |   P5   |
+-----+-----+-----+-----+-----+
0         5         9        11        17        20
=====
```

2. SJF (Non-Preemptive) Scheduling Output

Execution Order and Time Slices:

```
=== Gantt Chart ===
+-----+-----+-----+-----+-----+
|   P1   |   P3   |   P5   |   P2   |   P4   |
+-----+-----+-----+-----+-----+
0         5         7        10        14        20
=====
```

3. Priority Scheduling (Preemptive)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+-----+-----+-----+-----+-----+
|   P1   |   P2   |   P4   |   P3   |   P5   |
+-----+-----+-----+-----+-----+
0         5         9        15        17        20
=====
```

4. Pre-emptive SJF Scheduling (Shortest Remaining Time First)

Execution Order and Time Slices:

```
=== Gantt Chart ===
+-----+-----+-----+-----+-----+
|  P1  |  P3  |  P1  |  P5  |  P2  |  P4  |
+-----+-----+-----+-----+-----+
0       2       4       7      10      14      20
=====
```

5. Round Robin

Execution Order and Time Slices:

```
=== Gantt Chart ===
+-----+-----+-----+-----+-----+-----+-----+-----+
|  P1  |  P2  |  P3  |  P1  |  P4  |  P5  |  P2  |  P1  |  P4  |  P5  |  P4  |
+-----+-----+-----+-----+-----+-----+-----+-----+
0       2       4       6       8      10      12      14      15      17      18      20
=====
```

Part B: Advanced FCFS Scheduling (with CPU Utilization and Idle Time)

Description: This enhanced version of FCFS scheduling includes additional performance metrics like CPU idle time, CPU utilization, and throughput. It handles cases where processes arrive late, causing idle periods in the CPU.

Logic:

1. Input number of processes and their AT and BT.
2. Sort processes by AT.
3. For each process:
 - **Start Time (ST)** = $\max(AT, \text{previous CT})$
 - **Completion Time (CT)** = $ST + BT$
 - **Turnaround Time (TAT)** = $CT - AT$
 - **Waiting Time (WT)** = $TAT - BT$
4. Calculate:
 - **Total Idle Time** = sum of all CPU gaps
 - **CPU Utilization (%)** = $\frac{(TotalTime - IdleTime)}{TotalTime} \times 100$
 - **Throughput** = $\frac{No. of Processes}{TotalTime}$

Example Calculation (FCFS)

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	4	9	8	4
P3	2	2	11	9	7
P4	3	6	17	14	8
P5	4	3	20	16	13

Average Waiting Time:

$$\frac{0 + 4 + 7 + 8 + 13}{5} = 6.4$$

Average Turnaround Time:

$$\frac{5 + 8 + 9 + 14 + 16}{5} = 10.4$$

CPU Utilization and Throughput

Total Execution Time = 20

Idle Time = 0

CPU Utilization:

$$\frac{20 - 0}{20} \times 100 = 100\%$$

Throughput:

$$\frac{5}{20} = 0.25 \text{ processes per unit time}$$